arXiv:2409.05177v1 [cs.SE] 8 Sep 2024

Insights from Benchmarking Frontier Language Models on Web App Code Generation

Yi Cui
ONEKQ Lab
yi@onekq.ai

Abstract

This paper presents insights from evaluating 16 frontier large language models (LLMs) on the WebApp1K benchmark, a test suite designed to assess the ability of LLMs to generate web application code. The results reveal that while all models possess similar underlying knowledge, their performance is differentiated by the frequency of mistakes they make. By analyzing lines of code (LOC) and failure distributions, we find that writing correct code is more complex than generating incorrect code. Furthermore, prompt engineering shows limited efficacy in reducing errors beyond specific cases. These findings suggest that further advancements in coding LLM should emphasize on model reliability and mistake minimization.

1 Introduction

In [Cui, 2024], we introduced WebApp1K, a benchmark to evaluate web app code generation performance of frontier LLMs. The performance results of these 16 models are summarized in Tab. 1.

Model	pass@1	pass@5	pass@10
gpt-4o-2024-08-06	0.885	0.9047	0.909
claude-3.5-sonnet	0.8808	0.8845	0.886
gpt-4o-2024-05-13	0.8702	0.9013	0.909
gpt-4o-mini	0.8271	0.8534	0.858
mistral-large-2	0.7804	0.8191	0.831
deepseek-coder-v2-instruct	0.7002	0.8009	0.827
gemini-1.5-pro	0.6813	0.7678	0.795
gemini-1.5-flash	0.57	0.6427	0.663
deepseek-coder-v2-lite-instruct	0.4606	0.6144	0.653
mixtral-8x22b-instruct	0.3074	0.4821	0.533
llama-v3-70b-instruct	0.3323	0.4462	0.489
llama-v3p1-405b-instruct	0.302	0.4053	0.437
llama-v3p1-8b-instruct	0.2512	0.3941	0.432
llama-v3p1-70b-instruct	0.1027	0.1848	0.246
mixtral-8x7b-instruct	0.1269	0.196	0.218
llama-v3-8b-instruct	0.0679	0.1183	0.139

Table 1: pass@k results for frontier LLMs

In this report, we share insights gained from the code written by these 16 models To prevent benchmark contamination, we do not reveal the actual code, but their outcome aggregated by certain measures. The artifacts are on GitHub and Huggingface: the dataset containing all 1000 problems of WebApp1K[Lab, 2024a], the script[Lab, 2024c] to run WebApp1K, and the leaderboard[Lab, 2024b].

The rest of this report is organized as follows. Sec. 2 reveals the difficulty of WebApp1K to each model in terms of test failures. Sec. 3 analyzes the LOC (lines of code) distributions. Sec. 4 provides a deep dive to errors made by models. Sec. 5 presents related works. Sec. 6 concludes our findings and discusses future directions.

2 Benchmark Difficulty

2.1 Failure Distributions

We begin by examining the difficulty of the benchmark. We made each model solve each coding problem for 10 times, which gives us 160 solutions per problem. If a solution passes the tests, it is considered a succes, otherwise a failure. Fig. 1 shows number of failures per problem. The more failures a problem collects, the more difficult it is.

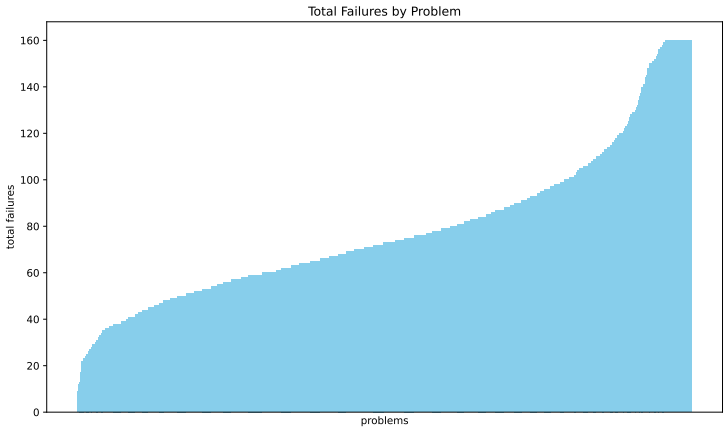


Figure 1: Failures per Problem

As the figure shows, the majority of the problems are situated on the left side of the graph, characterized by low failure rates, indicating that these problems were relatively easy, especially for top-ranked models. Conversely, a small cluster of problems on the far right exhibit extremely high failure rates. The most difficult 38 problems have never been solved by any model. This pattern still holds after we break down Fig. 1 by applications. More details are available in Appendix A.

2.2 Does the Code Build?

Of the total 160,000 solutions produced (10 runs by each of the 16 models for 1000 problems), only 172 have syntax errors, i.e. the build failure rate is 0.1%. In particular, the solutions by Claude 3.5 and Mistral Large 2 have no syntax errors.

This means all models are able to write high-quality and compilable code. Yet some of them did not manage to meet test expectations, some explicit and others implicit, therefore causing the failures.

3 Line-of-Code (LOC) Analysis

Thanks to the modularized design of the React framework, the solutions output by all models universally follow the template outlined in Tab. 2, with no need for any explicit prompting. As such, we seek to another proxy signal, LOC (line-of-code), to gain insights.

3.1 LOC Distribution by Models

In Tab. 3, we rank models by their median LOC alongside their respective $pass@1$ scores. Picking one $pass@k$ is sufficient because all scores produced basically the same model rankings as shown in Tab. 1.

```
// Import Statements
import React from 'react';

// Component Declaration
const componentName = (...) -> {
  // function body

  // JSX-based UI layout
  return (
    <div> ... </div>
  );
};

// Export Statements
export default componentName;
```

Table 2: A React Solution to a WebApp1K Problem

Model	**Median LOC**	**pass@1**
mixtral-8x7b-instruct	35	0.1269
llama-v3-8b-instruct	39	0.0679
gpt-4o-2024-05-13	39	0.8702
llama-v3p1-405b-instruct	40	0.3020
gpt-4o-2024-08-06	40	0.8850
deepseek-coder-v2-instruct	40	0.7002
gpt-4o-mini	40	0.8271
mistral-large-2	41	0.7804
gemini-1.5-flash	41	0.5700
llama-v3p1-8b-instruct	42	0.2512
mixtral-8x22b-instruct	43	0.3074
claude-3.5-sonnet	43	0.8808
llama-v3-70b-instruct	43	0.3323
deepseek-coder-v2-lite-instruct	43	0.4606
gemini-1.5-pro	45	0.6813
llama-v3p1-70b-instruct	46	0.1027

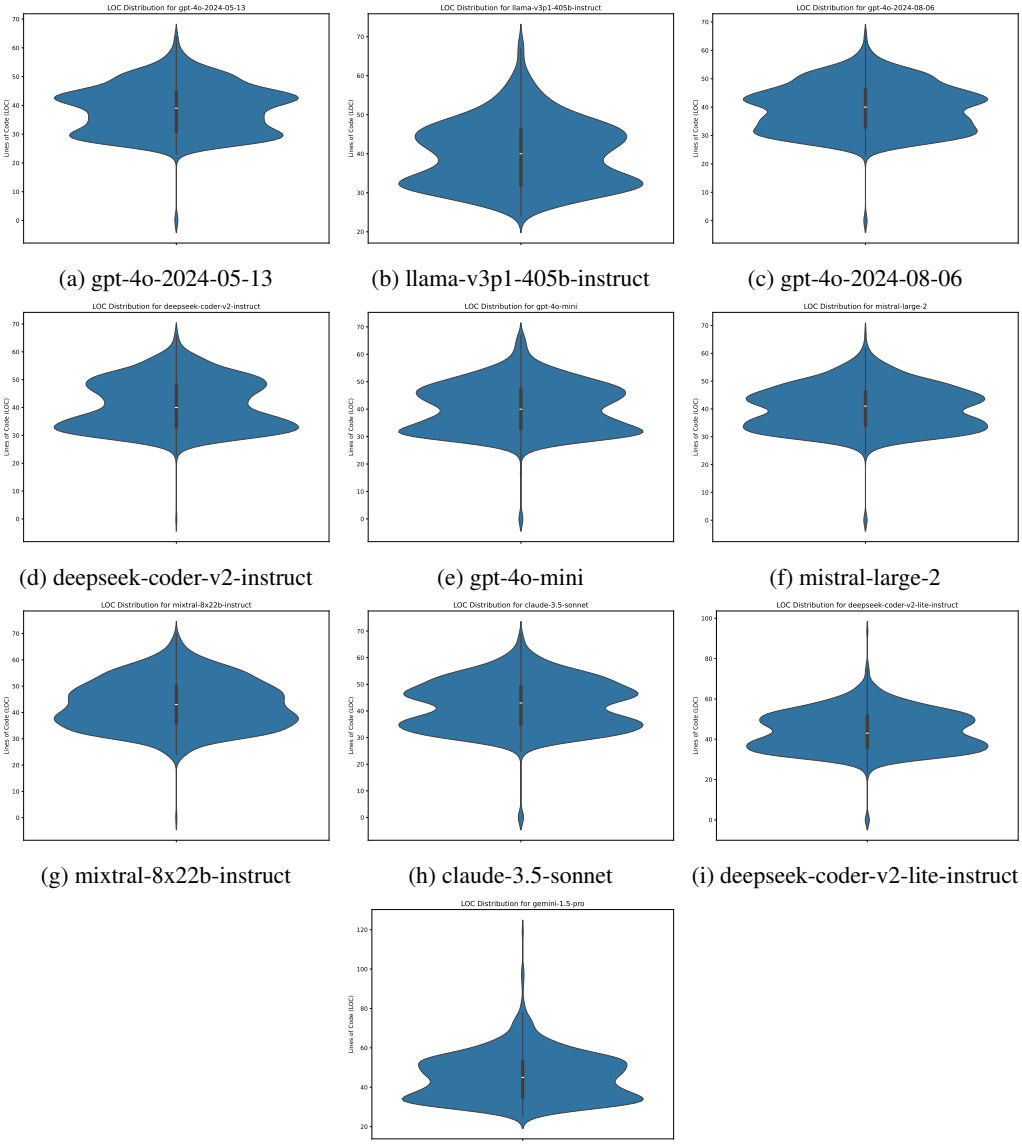
Table 3: Models Ranked by Median LOC with $pass@1$

We observe that the median LOCs across all models stay close, ranging from 35 to 46. We believe this narrow range is largely enforced by the conciseness and expressiveness of the React framework itself. Also there is no strong correlation between the conciseness (median LOC) and correctness ($pass@1$). For example, mixtral-8x7b-instruct, which has the shortest median LOC, ranks quite low on $pass@1$ (0.1269). Conversely, stronger models like claude-3.5-sonnet and gpt-4o-2024-08-06, generate longer code. Other models, e.g. deepseek-coder-v2-instruct and gemini-1.5-pro, strike a balance between median.

Next, we use violin charts to visualize LOC distribution of each model. The distributions are either bimodal or unimodal, and they are collected in Fig. 2 and Fig. 3 respectively.

Notably, all high-performing models with high $pass@1$ scores are located in Fig. 2. These models, such as the gpt-4o variants and deepseek-coder series, demonstrate higher variability in their LOC distributions, i.e. bimodal. The two distinct peaks in these models' distributions suggests that they generate both shorter and longer code lengths, depending on the task. Importantly, the median LOC values for these bimodal models consistently fall between the two peaks, highlighting a balance in their code generation. Also the higher of the two peaks often corresponds to smaller LOC. This suggests that while these models can produce longer code when necessary, they tend to generate shorter, more optimized code in most cases.

In contrast, Fig. 3 contains smaller models. Some exhibit near-perfect normal distributions, e.g. mixtral-8x7b-instruct and llama-v3-8b-instruct. These models generate LOC distributions that are tightly centered around their medians, indicating more consistent and predictable behavior. The lack of bimodal characteristics in these distributions reflects a more stable output across tasks, but with lower complexity compared to the larger models in Fig. 2.



(a) gpt-4o-2024-05-13 (b) llama-v3p1-405b-instruct (c) gpt-4o-2024-08-06

(d) deepseek-coder-v2-instruct (e) gpt-4o-mini (f) mistral-large-2

(g) mixtral-8x22b-instruct (h) claude-3.5-sonnet (i) deepseek-coder-v2-lite-instruct

(j) gemini-1.5-pro

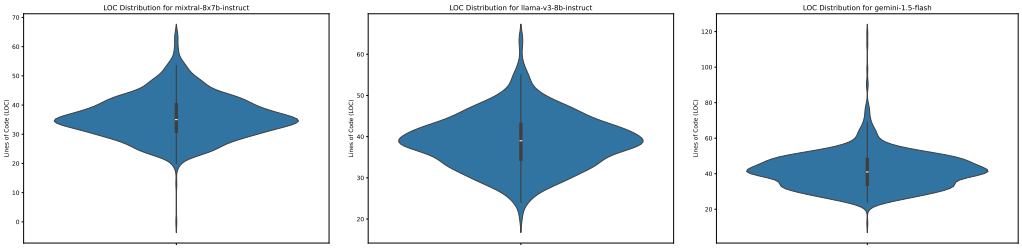
Figure 2: LOC Distribution by Model (BiModal)

We also study LOC distribution sharded by applications and have the similar observations. More details can be found in Appendix B.

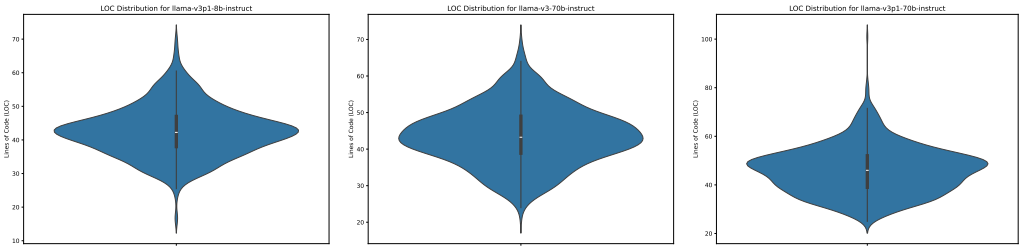
3.2 Impact of Success/Failure

To get more insights, we search for statistical distinction between successful model outputs and failed outputs. In Fig. 4 and 5, we visualize the LOC distribution separately for succssful outputs and failed ones, for each model. The graphs are ranked by $pass@1$, where higher $pass@1$ means bigger success sample set and smaller failure sample set. We normalize the width of each violin chart by its sample set size, hence resulting in the thinnest failure graph for the model with the highest $pass@1$. The graph gradually grows wider as the model performance degrades. The opposite pattern is observed for the success violin chart.

An important finding here is that the success distribution is always more complex than its failure counterpart, with more peaks and deviations. Fig. 5 groups lower performing models whose failure

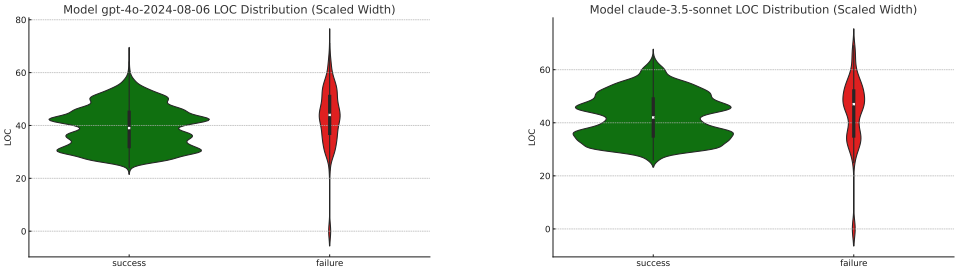


(a) mixtral-8x7b-instruct (b) llama-v3-8b-instruct (c) gemini-1.5-flash

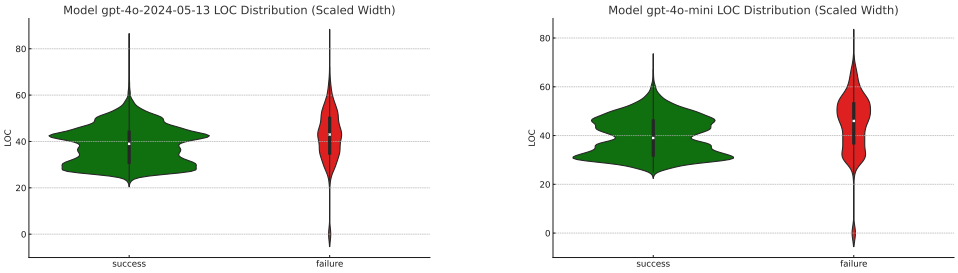


(d) llama-v3p1-8b-instruct (e) llama-v3-70b-instruct (f) llama-v3p1-70b-instruct

Figure 3: LOC Distribution by Model (UniModal)



(a) gpt-4o-2024-08-06 ($pass@1$ = 0.885) (b) claude-3.5-sonnet ($pass@1$ = 0.8808)

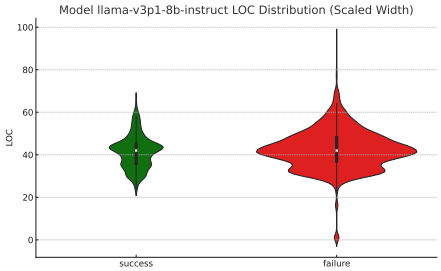


(c) gpt-4o-2024-05-13 ($pass@1$ = 0.8702) (d) gpt-4o-mini ($pass@1$ = 0.8271)

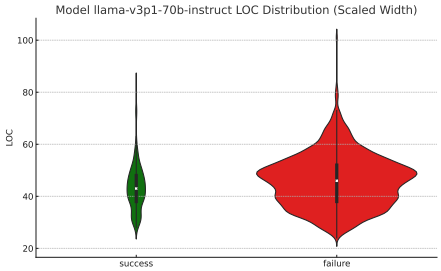
Figure 4: LOC Distribution by Model of High $pass@1$: Success vs Failure

sample set dominates the success sample set. The failure LOC distributions are unimodal, in contrast with the multimodal distributions of top models in Fig. 4. This implies the inherent complexity involved in writing correct code even when the mean LOC is less than 50.

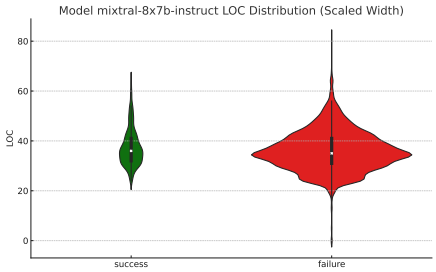
The success/failure LOC distributions of remaining 8 models are collected in Appendix C. Also Appendix D shows LOC distributions sharded by applications.



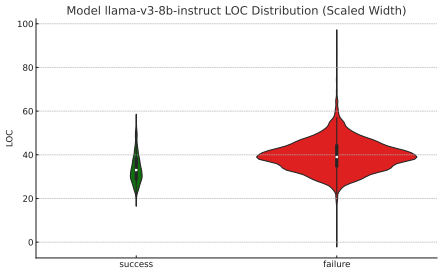
(a) llama-v3p1-8b-instruct (pass@1 = 0.2512)



(b) llama-v3p1-70b-instruct (pass@1 = 0.1027)



(c) mixtral-8x7b-instruct (pass@1 = 0.1269)



(d) llama-v3-8b-instruct (pass@1 = 0.0679)

Figure 5: LOC Distribution by Model of Low $pass@1$: Success vs Failure

4 Error Analysis

When testing model-generated code, the failed solutions end up with error logs, one log file for each JS file. In this section, we study these logs and share our findings.

4.1 Error Types

There are seven types of errors, which we code them to A through G. They are summarized in Tab. 4.

Error Code	Name	Verbatim Error	Root Cause
A	Version Mismatch	TypeError	Deprecated framework functions are used
B	Text Mismatching	TestingLibrary ElementError	Attributes or texts of HTML tags do not match test expectations
C	API Call Mismatch	expect(received)	Mock APIs are called less or more than expected
D	Uninstalled Module	Cannot find module	Imported module is not installed
E	Invalid API Call	fetch-mock	The call signature does not match the test expectation
F	Scope Violation	ReferenceError	An out-of-scope call is made to a locally-defined function
G	Missing UI Element	Element type is invalid	No UI element is defined in the code

Table 4: Error Table

The verbatim errors are the original error messages or codes captured by the log. Each of them is broadly scoped to contain a wide array of behaviors. However, in the context of our benchmark, we find all verbatim errors are projected to narrowband of behaviors attributed to the same root causes.

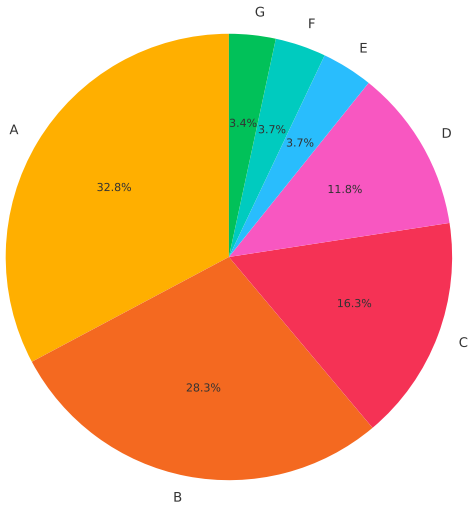


Figure 6: Error Type Distribution

Fig. 6 shows the error type distribution. Note that the same error type can appear in the same log multiple times because for each benchmark challenge, the code needs to pass two unit tests, each containing multiple expectations.

4.2 Singular and Twin Errors

An error log can contain a combination of many error types, indicating the code is poorly implemented. But this is not the dominant pattern. 93% of error logs contain either a singular error or twin errors. Fig. 7 shows the distribution of singular and twin errors.

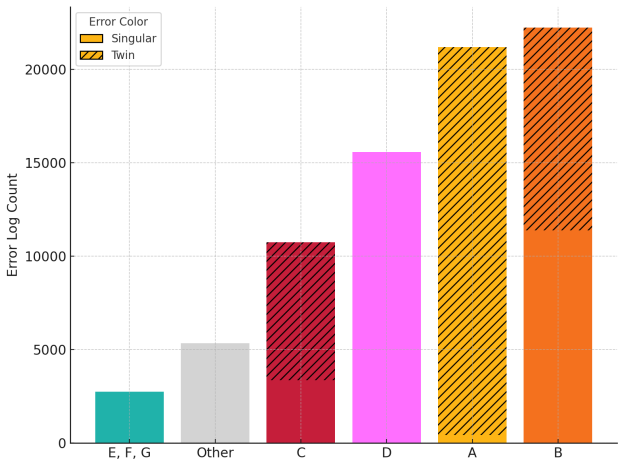


Figure 7: Distribution of Singular and Twin Errors

Singular error means the log contains only one error pointing to a single line. Twin errors are two errors of the same type, preeminently pointing to the same error line. Since the code needs to pass two unit tests, often times the same bug offends both tests. This means that even upon failures, all models produce quality code, but with only one bug.

4.3 Error Distribution by Models

In Fig. 8, we show the error distribution separately for each model. The most important finding here is that no model is immune to any of the seven error types, even when the raw error counts differ by one order of magnitude bewteen two extremes.

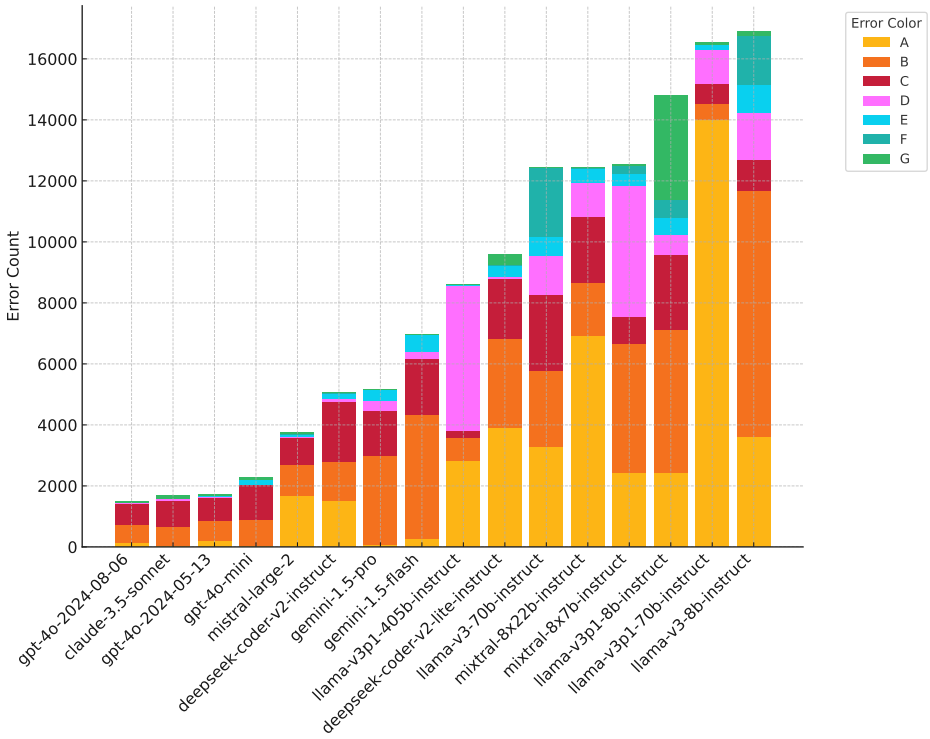


Figure 8: Error Distribution by Models

This means that all models possess the same knowledge and capabilities to write high-quality code which meets test expectations, and same inherent vulnerabilities resulting in the same types of errors. But top models distinguish themselves at lower error rates, i.e. ability to make fewer bugs.

We also study error distribution sharded by applications, whose results can be found in Appendix E.

4.4 Targeted Prompt Optimization

Prompt engineering is a common practice to improve model outputs by paying additional efforts to refine model inputs. After root causing each error type, we apply targeted prompt optimization to see if it can help weaker models to reduce bugs in their outputs.

Specifically, following the standard prompt below, we add a sentence reminding the model to avoid a specific type of errors.

$$\text{Generate } \{file_name\} \text{ to pass the tests below:} \\ \{success_test_code\}\{failure_test_code\}. \text{ RETURN CODE ONLY.} \tag{1}$$

Type A is the only error type we manage to achieve significant improvement. To give more background, $useHistory$ is a commonly used framework function deprecated in React v6, 2021. As such, type A error is triggered each time $useHistory$ appears in the code.

We choose llama-v3p1-70b-instruct because it makes the most type A errors (Fig. 8) among all models. We use different prompts to emphasize to the model not to call $useHistory$. As shown in Tab. 5, the more specific the prompt for the model to follow, the better the error reduction.

Prompt	Specificity	Error Reduction
USE useNavigate (React Router v6), NOT useHistory (React Router v5).	Functions	100%
Use React Router v6, not v5.	Packages and versions	100%
If you call useHistory (React Router v5), REPLACE it with useNavigate (React Router v6).	Functions	99.93%
Use the LATEST VERSION of React and React Router.	Packages only	85.87%
Use the LATEST VERSION for all packages.	Too many packages	**0%**

Table 5: Prompts to Reduce Type A Errors (llama-v3p1-70b-instruct)

Tab. 6 shows prompts targeted for error types B, C, and D, as well as the model making the most errors. However, none of them can reduce errors by more than 5% because of inherent challenges to make the prompt more specific.

Error Type	Model	Prompt	Lack of Specificity
B	llama-v3-8b-instruct	Make sure the text or test-id of UI elements match the test code.	test-id and text are different per test.
C	llama-v3p1-8b-instruct	Trigger mocked API EXACTLY ONCE. Avoid no trigger and duplicate trigger which might be caused by useEffect.	API name is different per test.
D	llama-v3p1-8b-instruct	ONLY USE modules appeared in test files.	Too many modules to enumerate.

Table 6: Prompts to Reduce Type B, C, D Errors

We skip experiments for error types E, F, and G, because of their relatively small sample sizes.

5 Related Works

The development and evaluation of large language models (LLMs) for code generation have been an area of significant research interest in recent years.

5.1 Benchmarks

Benchmarks are essential for evaluating the effectiveness and generalizability of models across various software engineering tasks. CodeSearchNet[Husain et al., 2020] is a benchmark to evaluate semantic code search performance. HumanEvalPack[Muennighoff et al., 2024] assesses the ability of models like Codex to generate correct Python code from natural language prompts. Defects4J[Just et al., 2014] is widely used to assess LLMs' ability to handle bug localization and repair in Java. XL-CoST[Zhu et al., 2022] evaluates how well LLMs can work across different programming languages. BugsJS[Gyimesi et al., 2019] collects real-world JavaScript bugs and is used to evaluate LLM ability to detect and fix bugs in web applications. ClassEval[Du et al., 2023] evaluates class-level code generation.

5.2 Error Analysis

Error analysis is a critical area of research that focuses on understanding and improving the LLM weaknesses on software tasks. BugAID[Hanam et al., 2016] is a system to discover JavaScript bug patterns in JavaScript and web applications. DeepFix[Gupta et al., 2017] is a deep learning-based system repairing errors in C programs. TSSB-3M[Richter and Wehrheim, 2022] is a large dataset of single-statement bugs across multiple languages. ManySStuBs4J[Karampatsis and Sutton, 2020] is a dataset of bug-fix pairs commonly used to train models to detect subtle errors in Java.

5.3 Prompt Engineering

Quite a few studies focus on prompt engineering to improve LLM performance on coding tasks. In the era of GPT-3[Brown et al., 2020] and Codex[Chen et al., 2021], prompt engineering has been used for code translation tasks. Chain-of-Code[Li et al., 2024] expands on Chain-of-Thought by way of pseudocode. DotPrompts[Agrawal et al., 2023] leverages prompts for code summarization. APE (Automatic Prompt Engineer)[Zhou et al., 2023] automates prompt creation by exploring different configurations to LLM code repair performance.

5.4 Code Complexity

Code complexity research focus on understanding how models handles complex code and generates efficient code. CoCoNut[Lutellier et al., 2020] is a syntax-guided neural machine translation system for automatic program repair. AST-T5[Gong et al., 2024] incorporates Abstract Syntax Trees (AST) into T5 to understand the structure of complex code. InCoder[Fried et al., 2023] is a model designed for code generation and infilling tasks.

5.5 Error Reduction

Error reduction strategies focus on minimizing the number of mistakes made by models during code generation, completion, and repair tasks. CYCLE[Ding et al., 2024] is a self-refining model designed to reduce error rates by iterating outputs through code evaluation. CodeRL[Le et al., 2022] use reinforcement learning to reduces syntax errors via immediate feedback at training time. AlphaRepair[Xia and Zhang, 2022] incorporates static analysis feedback into zero-shot learning.

6 Conclusions and Future Works

In this report, we study WebApp1K results on 16 frontier LLMs, particularly failure rates, LOC distributions, and error types. Here are some tentative insights.

1. A failed solution is often one bug away from a correct one. This suggests that all models possess the necessary knowledge and capabilities, but mistake minimization is the key differentiator between top and weak models.
2. Success code outputs exhibit more complex patterns (LOC distribution) than failed code outputs, implifying more factors influencing the model output.
3. Prompt optimization is only effective when errors can be described (and hence avoided) in an exact and specific way.

We hope these insights are useful to the LLM community, especially model trainers. Below are some future tasks.

1. We will make the benchmark more challenging, forcing LLMs to write more lines of code to cover more scenarios.
2. We will incorporate more frameworks (e.g. Vue) and languages (e.g. Python) to increase the benchmark coverage.
3. We will continue to explore and evaluate new prompting techniques since they are crucial to LLM practitioners.

A Appendix: Benchmark Difficulty per Application

Fig. 9 shows the failure pattern broken down by applications.

1. *Consistency Across Applications*: All applications exhibit the same general shape—a large concentration of easier problems on the left side and a few harder problems on the right side. This consistency suggests that across different domains, there are always a few particularly challenging problems that models struggle with.

2. *Variations in Skewness*: Some applications, such as Fitness Tracking and Music Streaming, show a more pronounced skew with a sharp rise in failure rates for a few problems, indicating a steeper difficulty curve. Others have a more gradual increase, indicating a more even distribution of problem difficulty.
3. *Extreme Difficulty in Certain Applications*: Applications like Customer Support and Pet Care have a sharper increase towards the right, implying that these domains have a subset of problems that are especially challenging.
4. *Easier Applications*: In applications like Weather and Photo Gallery, the overall number of failures seems lower compared to other appli cations, suggesting that the problems in these areas were generally easier.

B Appendix: LOC Distribution by Applications

In Tab. 7, we rank median LOC for each application. Consistent with the case for model ranking (Tab. 3), the median values stay within a narrow range (37 to 46). This suggests that all models consistently produce solutions of similar length, irrespective of the task complexity or domain.

Application	Mean LOC
News Aggregator	37
Music Streaming	37
Online Marketplace	37
E-commerce	37
Recipe Sharing	38
Fitness Tracking	38
Online Learning	38
Blogging	39
Weather	40
Real Estate	42
Social Media	42
Job Board	42
Inventory Management	42
Pet Care	42
Travel Planning	42
Personal Finance	43
Customer Support	44
Photo Gallery	44
Event Management	45
Task Management	46

Table 7: Applications Ranked by Mean LOC

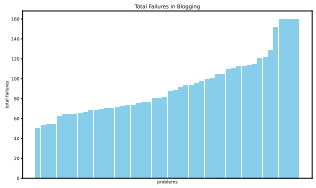
Fig. 10 collects violin charts of 14 applications following unimodal distribution, where the model outputs are centered around a common length, with less variation between extremes. The remaining 6 applications are in Fig. 11, following multimodal distribution. In both cases, the median LOC is always positioned centrally in each distribution, which suggests that the code generation is stable across applications. Applications in Fig. 11 exhibit more complex patterns, but the distributions remain balanced with the median value positioned at the center of the distribution.

C Appendix: LOC Distribution by Models: Success vs Failure

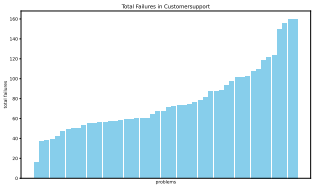
Continuing to Fig. 4 and 5, Fig. 12 shows the success/fail LOC distribution of remaining 8 models.

D Appendix: LOC Distribution by Applications: Success vs Failure

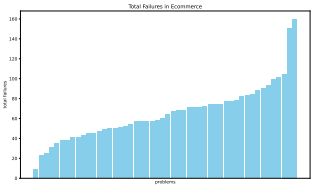
We conduct the same study described in Sec. 3.2, except we shard the LOC distribution across applications instead of models. The results are collected in Fig. 13.



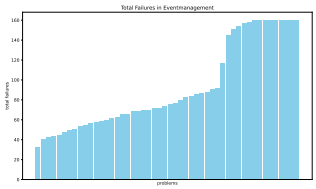
(a) Blogging



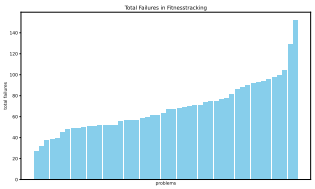
(b) Customer Support



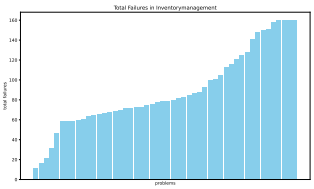
(c) E-commerce



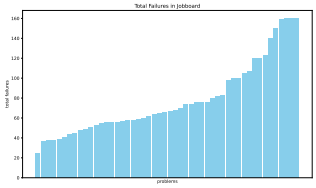
(d) Event Management



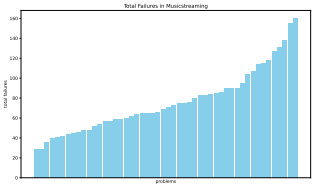
(e) Fitness Tracking



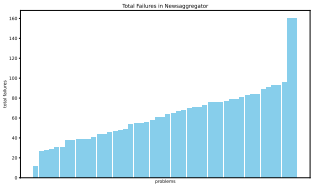
(f) Inventory Management



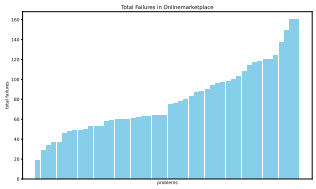
(g) Job Board



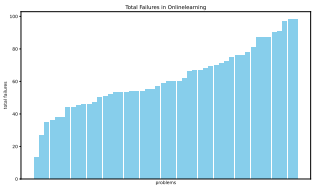
(h) Music Streaming



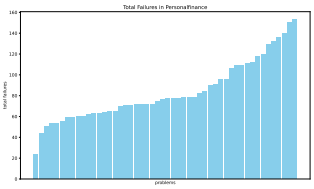
(i) News Aggregator



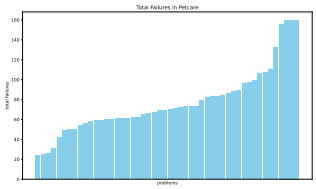
(j) Online Marketplace



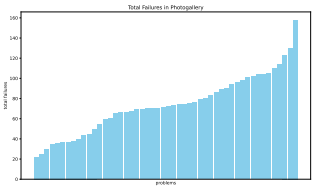
(k) Online Learning



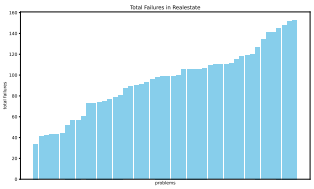
(l) Personal Finance



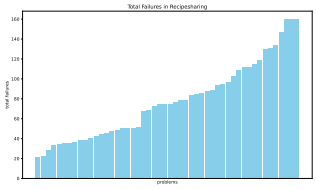
(m) Pet Care



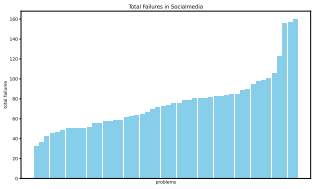
(n) Photo Gallery



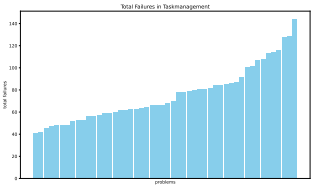
(o) Real Estate



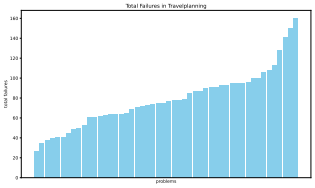
(p) Recipe Sharing



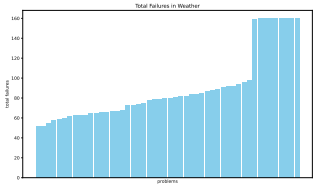
(q) Social Media



(r) Task Management

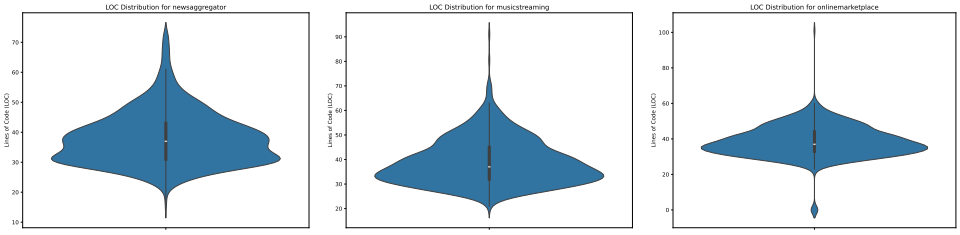


(s) Travel Planning



(t) Weather

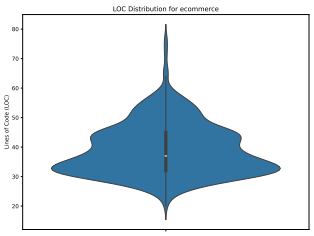
Figure 9: Failures per Problem by Application



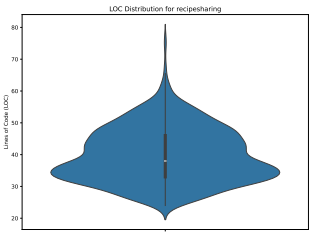
(a) News Aggregator

(b) Music Streaming

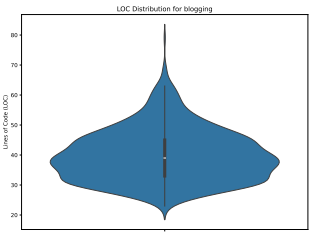
(c) Online Marketplace



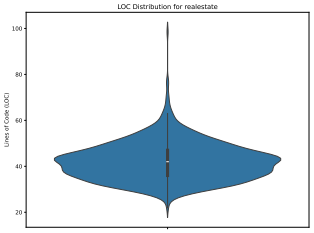
(d) E-commerce



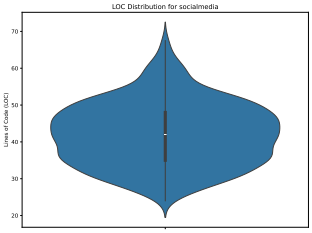
(e) Recipe Sharing



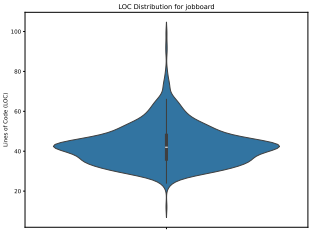
(f) Blogging



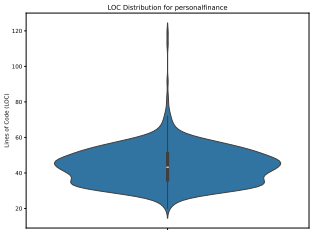
(g) Real Estate



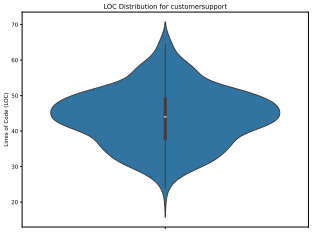
(h) Social Media



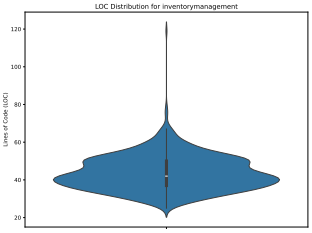
(i) Job Board



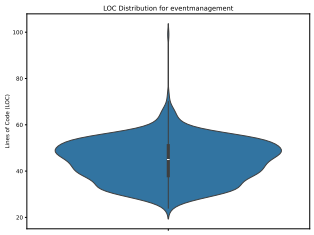
(j) Personal Finance



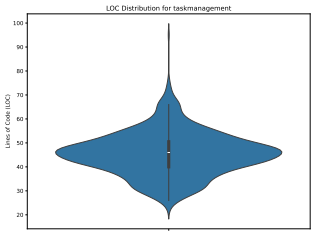
(k) Customer Support



(l) Inventory Management



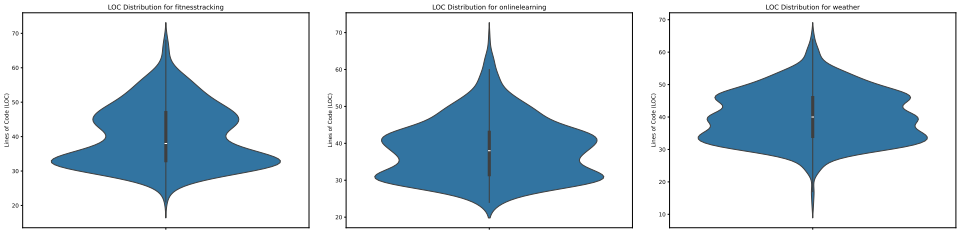
(m) Event Management



(n) Task Management

Figure 10: LOC Distribution by Applications: UniModal

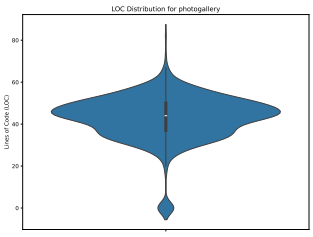
Since each application assembles outputs from all models with full spectrum of performances, the success and failure data set are about the equal size. Similar to what we have observed in model-based sharding (Sec. 3.2), the distribution pattern for success is equally or more complex than that for failure, summarized in Tab. 8.



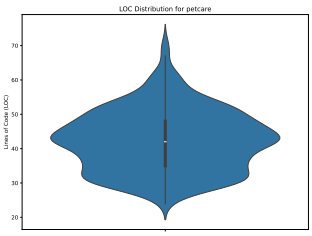
(a) Fitness Tracking

(b) Online Learning

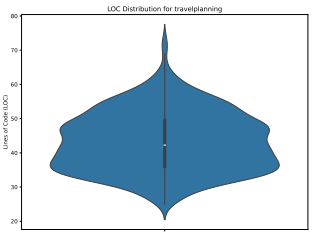
(c) Weather



(d) Photo Gallery



(e) Pet Care



(f) Travel Planning

Figure 11: LOC Distribution by Applications: MultiModal

	UniModal Success	**MultiModal Success**
UniModal Failure	(b) (q) (t)	(c) (d) (f) (g) (h) (j) (k) (l) (m) (n) (o) (p)
MultiModal Failure		(a) (e) (i) (r) (s)

Table 8: Summary of Fig. 13: Unimodal vs Multimodal

E Appendix: Error Distribution by Applications

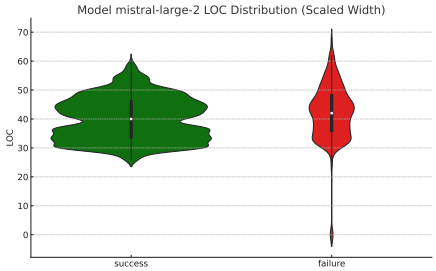
Fig. 14 shows error distribution by applications. Since each application assembles outputs from all models, the raw error counts are at the same scale for all applications. We do not find any distinctive patterns. There is neither special error nor special application.

References

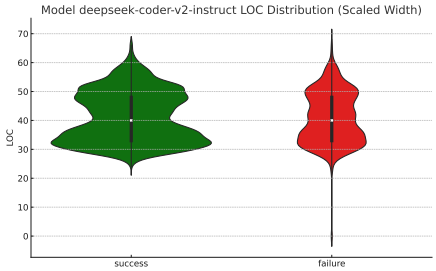
Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. Guiding language models of code with global context using monitors. `https://arxiv.org/abs/2306.10763`, 2023.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

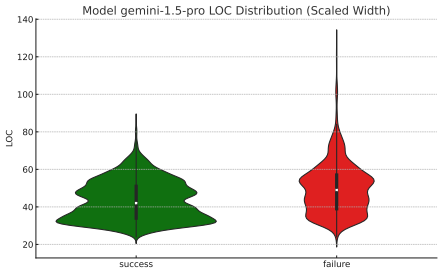
Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. `https://arxiv.org/abs/2107.03374`, 2021.



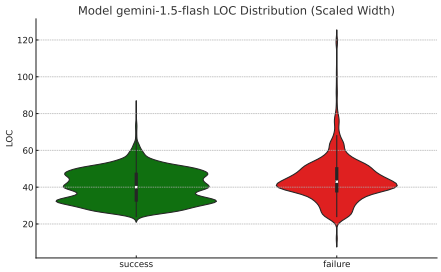
(a) mistral-large-2 ($pass@1$ = 0.7804)



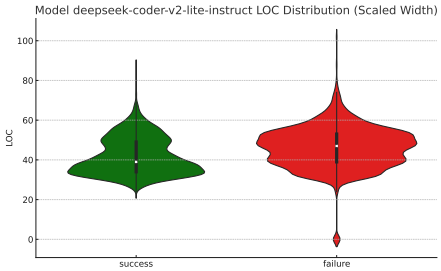
(b) deepseek-coder-v2-instruct ($pass@1$ = 0.7002)



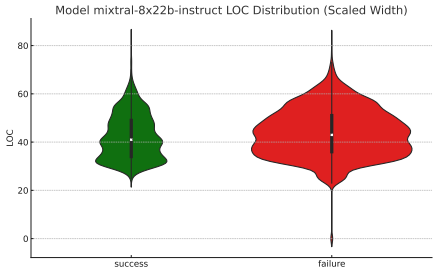
(c) gemini-1.5-pro ($pass@1$ = 0.6813)



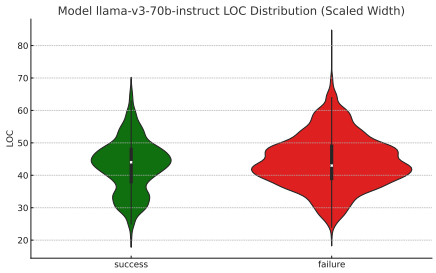
(d) gemini-1.5-flash ($pass@1$ = 0.57)



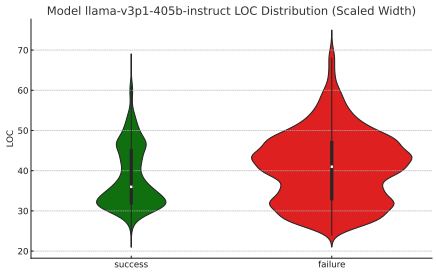
(e) deepseek-coder-v2-lite-instruct ($pass@1$ = 0.4606)



(f) mixtral-8x22b-instruct ($pass@1$ = 0.3074)



(g) llama-v3-70b-instruct (pass@1 = 0.3323)



(h) llama-v3p1-405b-instruct (pass@1 = 0.302)

Figure 12: LOC Distribution by Model: Success and Failure

Yi Cui. Webapp1k: A practical code-generation benchmark for web app development. `http://arxiv.org/abs/2408.00019`, 2024.

Yangruibo Ding, Marcus J. Min, Gail Kaiser, and Baishakhi Ray. Cycle: Learning to self-refine the code generation. `https://arxiv.org/abs/2403.18746`, 2024.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. Classeval: A manually-crafted benchmark for evaluating

llms on class-level code generation. https://arxiv.org/abs/2308.01861, 2023.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. https://arxiv.org/abs/2204.05999, 2023.

Linyuan Gong, Mostafa Elhoushi, and Alvin Cheung. Ast-t5: Structure-aware pretraining for code generation and understanding. https://arxiv.org/abs/2401.03003, 2024.

Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. Deepfix: Fixing common c language errors by deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.

Péter Gyimesi, Béla Vancsics, Andrea Stocco, Davood Mazinanian, Árpád Beszédes, Rudolf Ferenc, and Ali Mesbah. Bugsjs: a benchmark of javascript bugs. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, 2019.

Quinn Hanam, Fernando S. de M. Brito, and Ali Mesbah. Discovering bug patterns in javascript. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016.

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. https://arxiv.org/abs/1909.09436, 2020.

René Just, Darioush Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014.

Rafael-Michael Karampatsis and Charles Sutton. How often do single-statement bugs occur? the manysstubs4j dataset. In *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020.

ONEKQ Lab. Webapp1k dataset. https://huggingface.co/datasets/onekq-ai/WebApp1K-React, 2024a.

ONEKQ Lab. Webapp1k leaderboard. https://huggingface.co/spaces/onekq-ai/WebApp1K-models-leaderboard, 2024b.

ONEKQ Lab. Webapp1k github repo. https://github.com/onekq/WebApp1k, 2024c.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/8636419dea1aa9fbd25fc4248e702da4-Paper-Conference.pdf.

Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of code: Reasoning with a language model-augmented code emulator. https://arxiv.org/abs/2312.04474, 2024.

Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020.

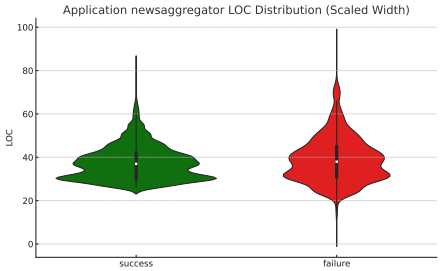
Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

Cedric Richter and Heike Wehrheim. Tssb-3m: mining single statement bugs at massive scale. In *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022.

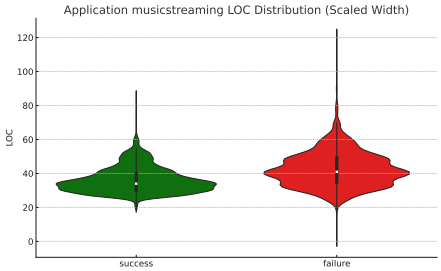
Chunqiu Steven Xia and Lingming Zhang. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022. URL https://doi.org/10.1145/3540250.3549101.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. https://arxiv.org/abs/2211.01910, 2023.

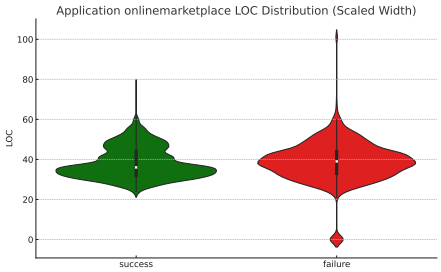
Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. Xlcost: A benchmark dataset for cross-lingual code intelligence. https://arxiv.org/abs/2206.08474, 2022.



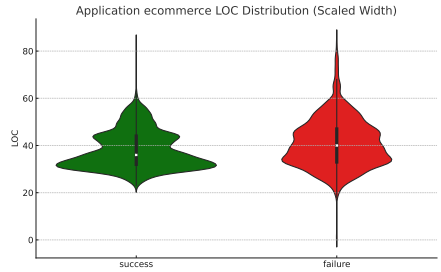
(a) News Aggregator (Mean LOC = 37)



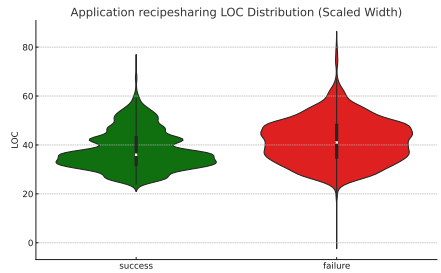
(b) Music Streaming (Mean LOC = 37)



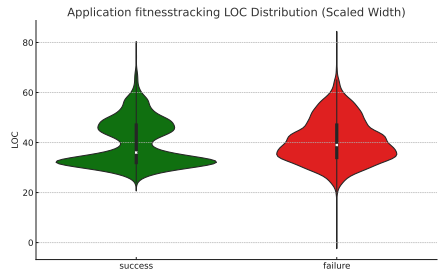
(c) Online Marketplace (Mean LOC = 37)



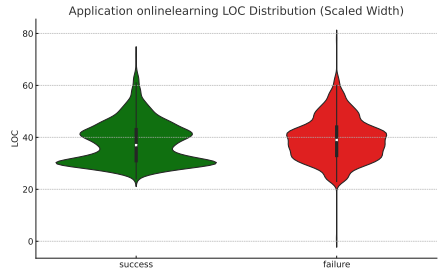
(d) E-commerce (Mean LOC = 37)



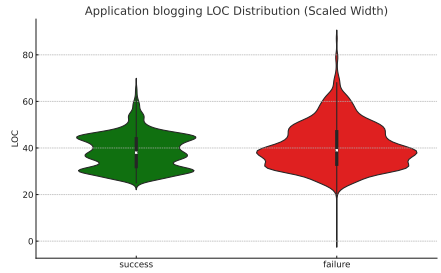
(e) Recipe Sharing (Mean LOC = 38)



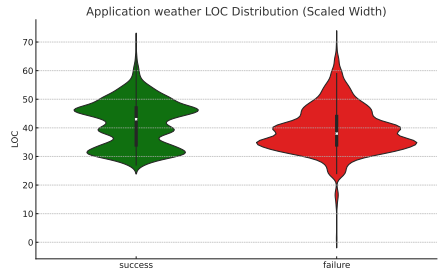
(f) Fitness Tracking (Mean LOC = 38)



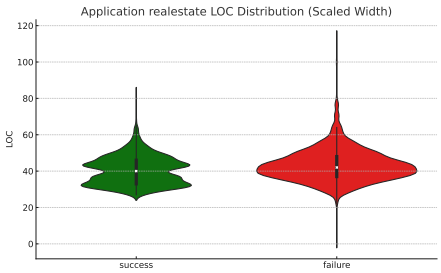
(g) Online Learning (Mean LOC = 38)



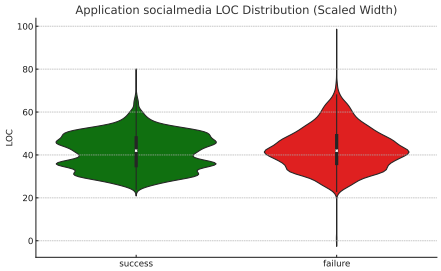
(h) Blogging (Mean LOC = 39)



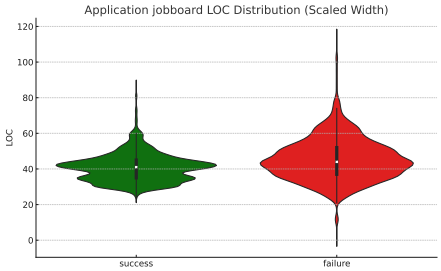
(i) Weather (Mean LOC = 40)



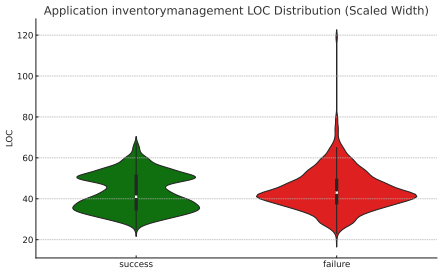
(j) Real Estate (Mean LOC = 42)



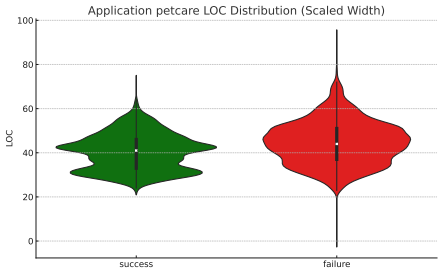
(k) Social Media (Mean LOC = 42)



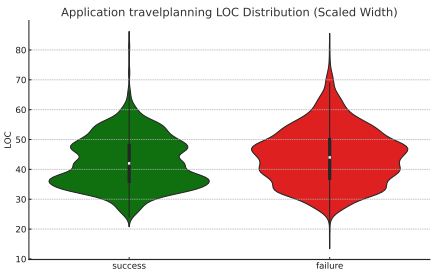
(l) Job Board (Mean LOC = 42)



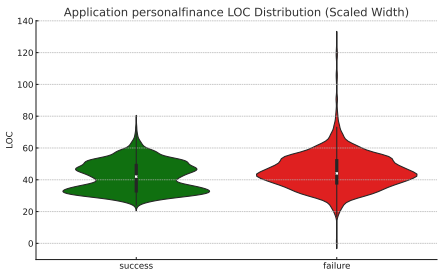
(m) Inventory Management (Mean LOC = 42)



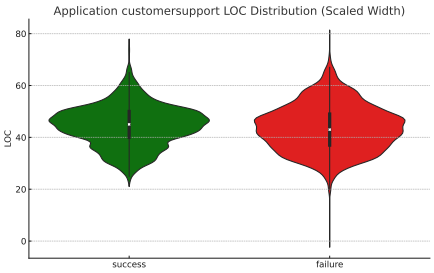
(n) Pet Care (Mean LOC = 42)



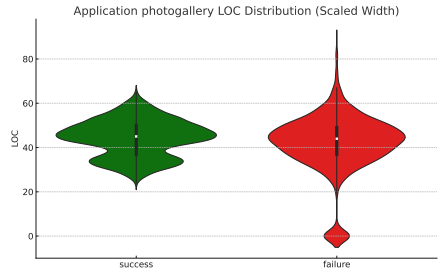
(o) Travel Planning (Mean LOC = 42)



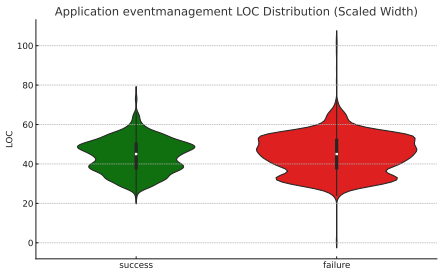
(p) Personal Finance (Mean LOC = 43)



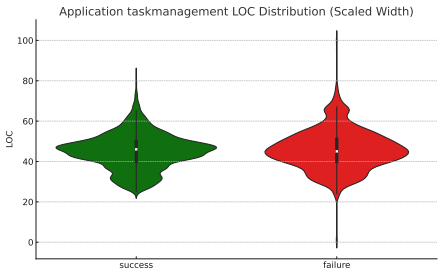
(q) Customer Support (Mean LOC = 44)



(r) Photo Gallery (Mean LOC = 44)



(s) Event Management (Mean LOC = 45)



(t) Task Management (Mean LOC = 46)

Figure 13: LOC Distribution by Application: Success vs Failure

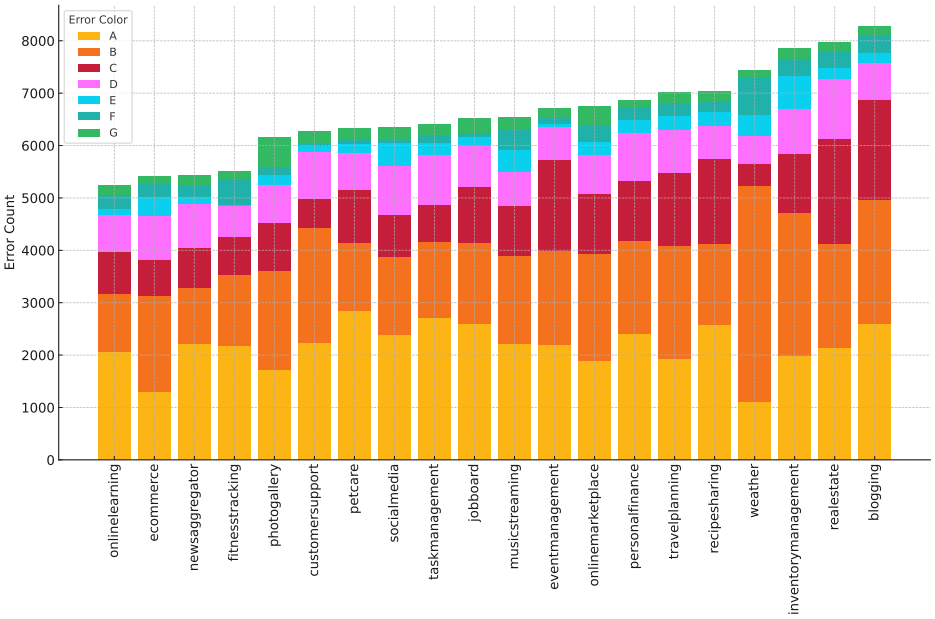


Figure 14: Errors by Applications